D A K A R  I N S T I T U T E  O F  T E C H N O L O G Y

# BIBLIOTHÈQUE NUMÉRIQUE DIT

## Rapport de Projet DevOps

Master 1 Intelligence Artificielle

Année académique 2025 – 2026


**G R O U P E  2**

AHOGA Josias · BAH Mamoudou · BAMBA Yannick · DIOP Seynabou

# Table des matières

1. Introduction ..... 3
2. Architecture du projet ..... 3
3. Technologies utilisées ..... 3
4. Microservices ..... 4
4.1 Service Livres ..... 4
4.2 Service Utilisateurs ..... 4
4.3 Service Emprunts ..... 4
5. Frontend Vue.js ..... 4
6. Base de données ..... 4
7. Fonctionnalités détaillées ..... 5
8. Guide d'utilisation de la plateforme ..... 6
8.1 Premier lancement ..... 6
8.2 Connexion et inscription ..... 10
8.3 Dashboard administrateur ..... 10
8.4 Gestion des livres ..... 12
8.5 Gestion des utilisateurs ..... 12
8.6 Gestion des emprunts ..... 13
8.7 Validation manuelle des emprunts ..... 13
8.8 Changement de mot de passe obligatoire ..... 14
9. Docker et Docker Compose ..... 15
10. Pipeline CI/CD Jenkins ..... 15
11. Conclusion ..... 17

# 1. Introduction

Ce rapport présente le projet de Bibliothèque Numérique développé dans le cadre du cours de DevOps du Master 1 en Intelligence Artificielle au Dakar Institute of Technology (DIT). Le projet consiste en une plateforme de gestion de ressources documentaires fondée sur une architecture microservices, permettant aux étudiants et enseignants du DIT d'emprunter des livres et à l'administrateur de gérer le catalogue, les utilisateurs et les emprunts.

L'objectif pédagogique est de mettre en pratique les concepts DevOps fondamentaux : conteneurisation avec Docker, orchestration avec Docker Compose, intégration continue avec Jenkins, et développement d'APIs REST avec FastAPI.

# 2. Architecture du projet

Le projet adopte une architecture microservices composée de cinq composants principaux qui communiquent entre eux via des appels HTTP internes (httpx) :

- **Service Livres (port 8001) :** CRUD du catalogue, gestion des exemplaires et de la disponibilité.
- **Service Utilisateurs (port 8002) :** authentification, gestion des comptes, création d'accès temporaires et désactivation de comptes.
- **Service Emprunts (port 8003) :** prêts, retours, validation manuelle ou automatique, détection des retards, limite de 2 emprunts par utilisateur.
- **Frontend Vue.js (port 80) :** interface web monofichier servie par Nginx, adaptée selon le rôle de l'utilisateur connecté.
- **PostgreSQL (port 5432) :** base de données unique partagée par les trois microservices.

# 3. Technologies utilisées

Le tableau ci-dessous récapitule les principales technologies et leurs rôles dans le projet :

| Technologie | Version | Utilisation |
|---|---|---|
| Python | 3.12 | Langage backend |
| FastAPI | 0.115 | Framework API REST |
| SQLAlchemy | 2.0 | ORM |
| PostgreSQL | 16 | Base de données relationnelle |
| Vue.js 3 | CDN | Framework frontend |
| Docker | Latest | Conteneurisation |
| Docker Compose | Latest | Orchestration des conteneurs |
| Jenkins | Latest | Pipeline CI/CD |
| Nginx | Alpine | Serveur web pour le frontend |
| httpx | 0.27 | Communication inter-services |

| | | |
|---|---|---|
| ReportLab | Latest | Génération de fichiers PDF |

# 4. Microservices

## 4.1 Service Livres

Ce service gère le catalogue de la bibliothèque. Chaque livre possède un titre, un auteur, un ISBN unique, une année de publication, un genre, une quantité totale d'exemplaires et un nombre d'exemplaires disponibles. Lors d'un emprunt, le nombre disponible est décrémenté ; lors d'un retour, il est incrémenté.

## 4.2 Service Utilisateurs

Ce service gère l'authentification et les comptes utilisateurs. Trois types de comptes sont pris en charge : étudiant (avec matricule au format NE2510081), professeur (avec pièce d'identité) et personnel administratif. L'authentification repose sur un email et un mot de passe hashé en SHA-256, avec un token JWT généré en interne.

Parmi les fonctionnalités clés, on trouve la création de comptes avec mot de passe temporaire (changement obligatoire à la première connexion) et la désactivation douce des comptes, qui préserve l'historique des emprunts.

## 4.3 Service Emprunts

Ce service gère les prêts de livres selon deux modes de validation. En mode automatique, l'emprunt est validé et le stock décrémenté immédiatement. En mode manuel, l'emprunt est créé avec le statut « En attente » sans toucher au stock, et l'administrateur doit le valider depuis l'interface.

La durée d'emprunt est fixée à 15 jours. Un utilisateur ne peut pas emprunter plus de 2 livres dans la même période : toute tentative d'emprunt supplémentaire est refusée par le système tant qu'un livre n'a pas été retourné. La détection automatique des retards est également intégrée.

# 5. Frontend Vue.js

L'interface est construite en tant que fichier HTML unique utilisant Vue.js 3 via CDN. Elle s'adapte dynamiquement au rôle de l'utilisateur connecté (administrateur ou membre). Le token d'authentification est transmis automatiquement dans le header Authorization de chaque requête HTTP vers les microservices.

# 6. Base de données

Une seule instance PostgreSQL est partagée par l'ensemble des microservices. Les tables sont créées automatiquement au démarrage grâce à SQLAlchemy :

| Table | Champs principaux |
|---|---|
| livres | id, titre, auteur, isbn, annee, genre, quantite, disponible |

| | |
|---|---|
| utilisateurs | id, nom, prenom, email, mot_de_passe, type, role, actif, mot_de_passe_temporaire |
| emprunts | id, utilisateur_id, livre_id, date_emprunt, date_retour_prevue, date_retour_reelle, retard, valide |
| config | cle, valeur (ex : auto_validation = true/false) |

## 7. Fonctionnalités détaillées

La plateforme offre un ensemble complet de fonctionnalités couvrant les besoins d'une bibliothèque académique :

- Authentification par email et mot de passe avec génération de token JWT.
- Inscription avec confirmation du mot de passe et validation conditionnelle (matricule étudiant, pièce d'identité pour les enseignants).
- Changement de mot de passe obligatoire à la première connexion (mot de passe temporaire).
- Désactivation douce des comptes membres avec conservation de l'historique d'emprunts.
- Blocage de la désactivation si des emprunts sont en cours.
- Gestion des livres avec suivi de la quantité d'exemplaires disponibles.
- Emprunt de livres avec durée fixe de 15 jours.
- Validation automatique ou manuelle des emprunts (toggle administrateur persisté en base).
- Maximum de 2 emprunts simultanés par utilisateur : l'utilisateur ne peut pas emprunter plus de 2 livres dans la même période et doit retourner un livre avant de pouvoir en emprunter un nouveau.
- Détection automatique des retards avec calcul du nombre de jours de dépassement.
- Tableau de bord adapté au rôle avec statistiques cliquables.
- Page Emprunts avec trois onglets : En cours, En attente et Tous.
- Recherche de livres par titre, auteur ou ISBN.
- Pipeline CI/CD Jenkins compatible Windows et Linux.

# 8. Guide d'utilisation de la plateforme

Cette section présente les principales fonctionnalités de la plateforme, illustrées par des captures d'écran de l'interface.

## 8.1 Premier lancement

Après avoir cloné le dépôt et lancé Docker Compose, la plateforme est accessible dans le navigateur :

```
git clone https://github.com/Sihiney/bibliotheque-dit.git
cd bibliotheque-dit
docker compose up --build
Ouvrir http://localhost dans le navigateur
```

Au premier lancement, aucun compte n'existe. Il faut créer le compte administrateur manuellement via l'API Swagger à http://localhost:8002/docs, en utilisant l'endpoint POST /utilisateurs avec le corps JSON suivant :

```
{"nom":"Admin","prenom":"DIT","email":"admin@dit.sn","mot_de_passe":"admin123","type":"
personnel_administratif","role":"admin"}
```

La capture ci-dessous montre l'interface Swagger UI du microservice Utilisateurs avec l'ensemble des endpoints disponibles :

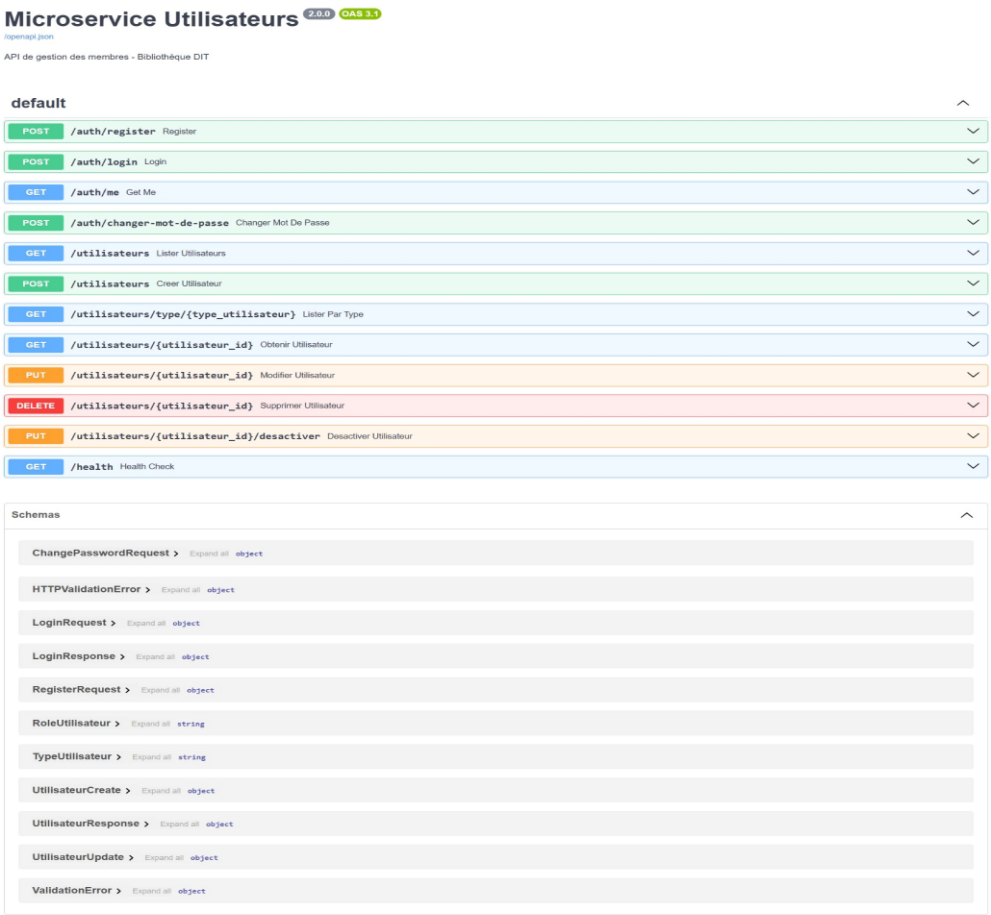


*Figure 1 – Swagger UI du microservice Utilisateurs*

Et voici l'exécution de la requête de création du compte administrateur via Swagger :

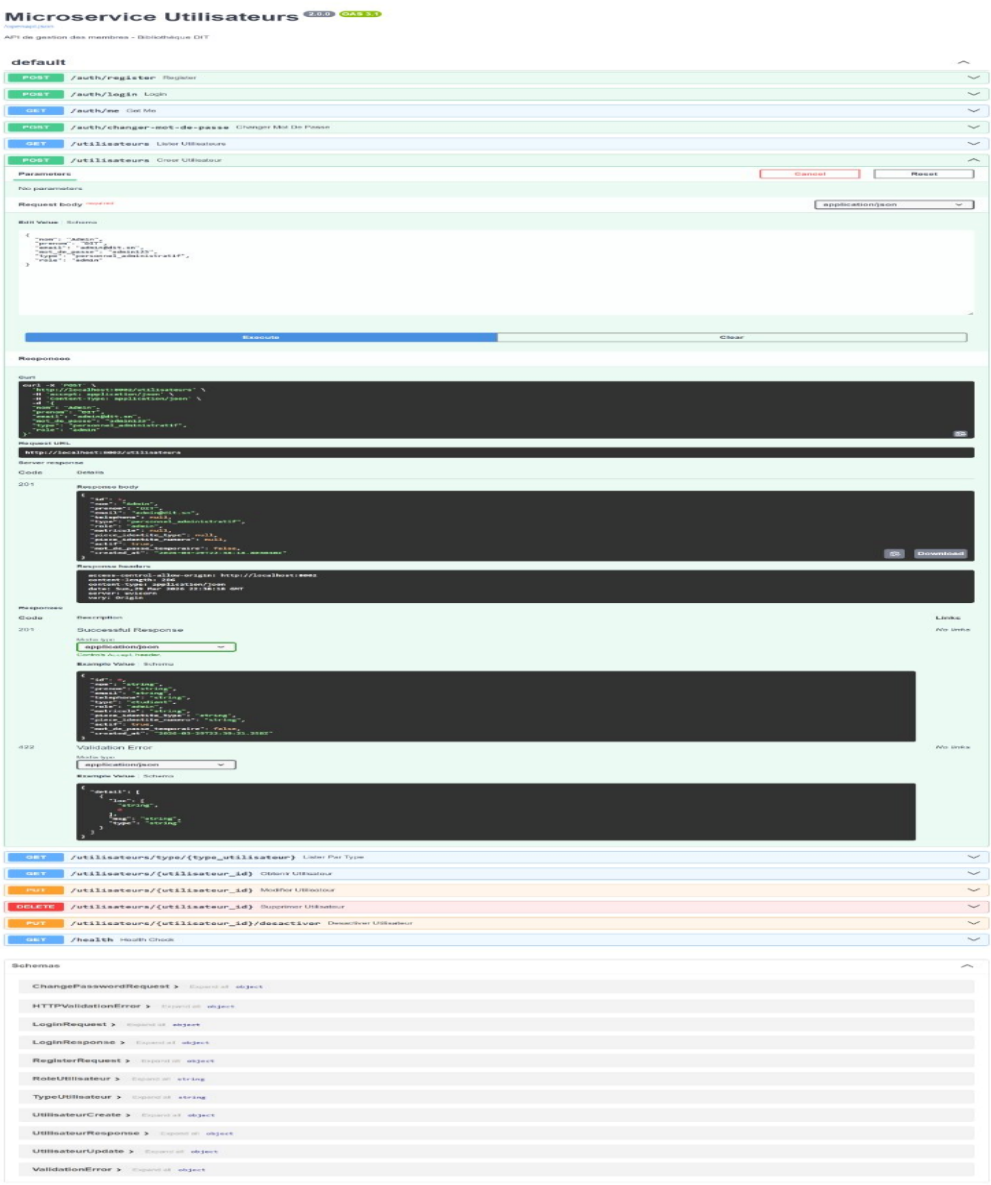


*Figure 2 – Création du compte administrateur via Swagger UI*

La capture ci-dessous montre l'interface Swagger UI du microservice Livres, avec les endpoints de gestion du catalogue (ajout, modification, suppression, recherche) :

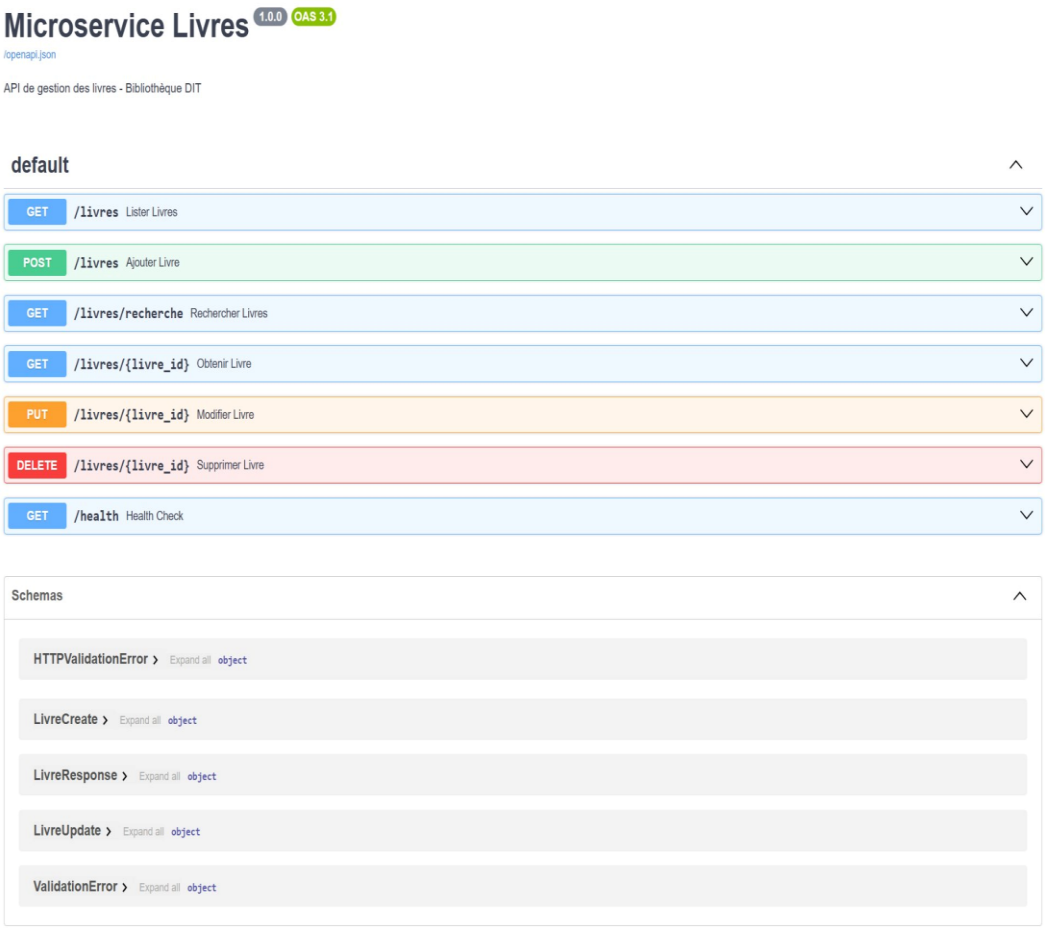


*Figure 3 – Swagger UI du microservice Livres*

La capture suivante présente le microservice Emprunts, qui expose les endpoints de gestion des prêts, retours, validation, détection des retards et configuration de la validation automatique :

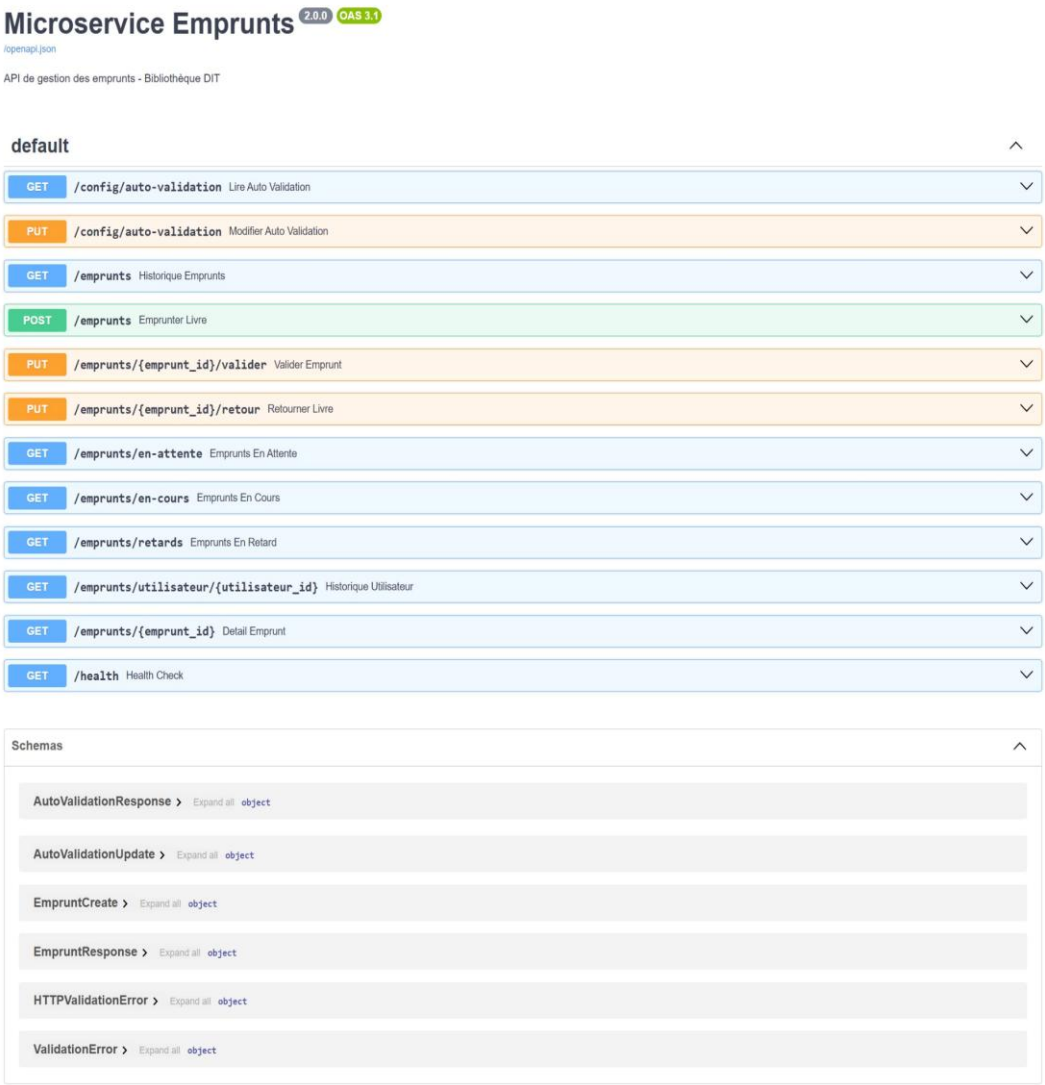


*Figure 4 – Swagger UI du microservice Emprunts*

## 8.2 Connexion et inscription

La page d'accueil présente le logo DIT à gauche et le formulaire de connexion/inscription à droite. L'utilisateur se connecte avec son email et son mot de passe. En cas d'oubli, un lien renvoie vers admin@dit.sn.

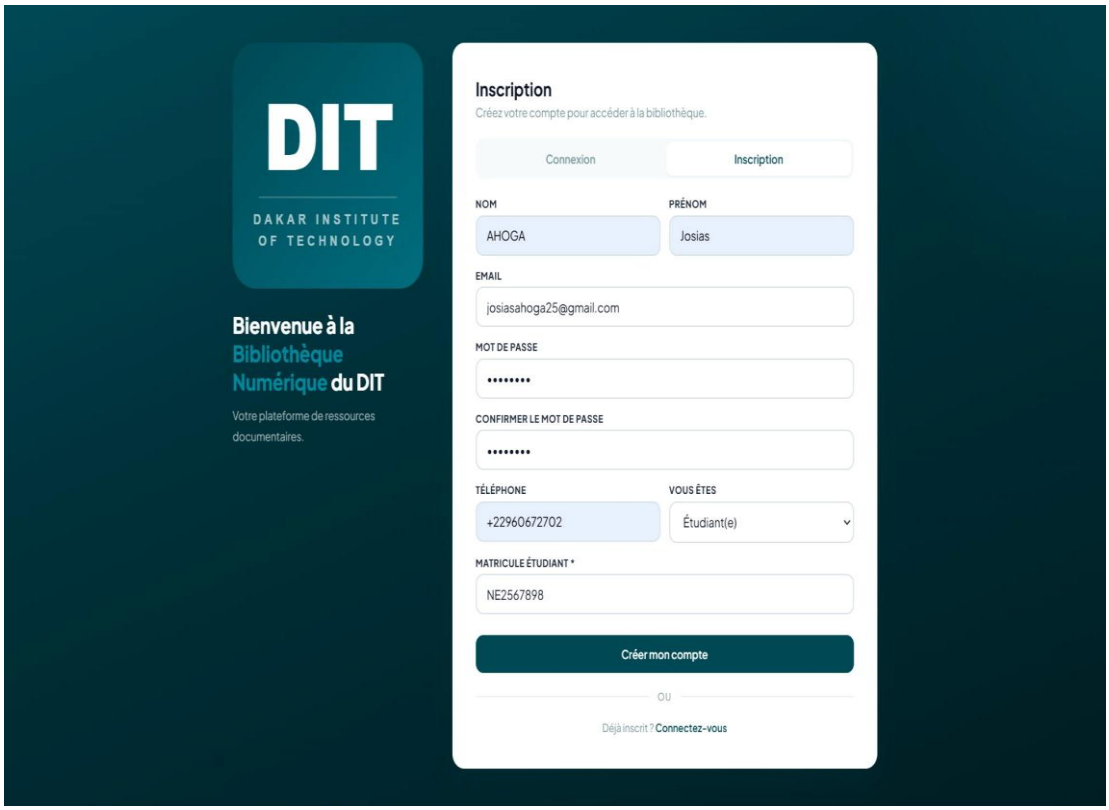


*Figure 5 – Page de connexion et d'inscription*

## 8.3 Dashboard administrateur

Le tableau de bord administrateur affiche cinq statistiques cliquables : Livres, Membres, Emprunts en cours, Retards et En attente de validation. La carte « En attente » affiche un badge rouge d'alerte lorsque des demandes sont en attente. Le toggle de validation automatique est branché sur l'API et persiste entre les sessions.

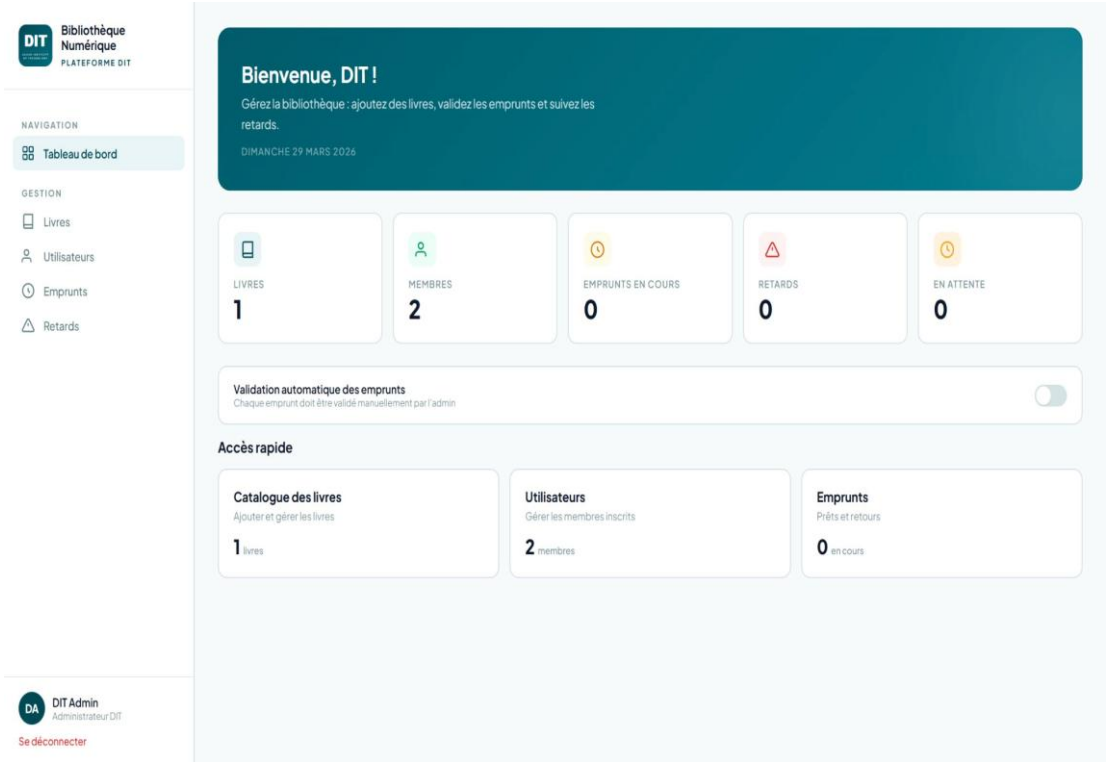


*Figure 6 – Dashboard administrateur avec statistiques et toggle de validation*

Le tableau de bord membre affiche uniquement les statistiques personnelles : livres disponibles, emprunts en cours et retards, ainsi que le tableau des emprunts actifs.

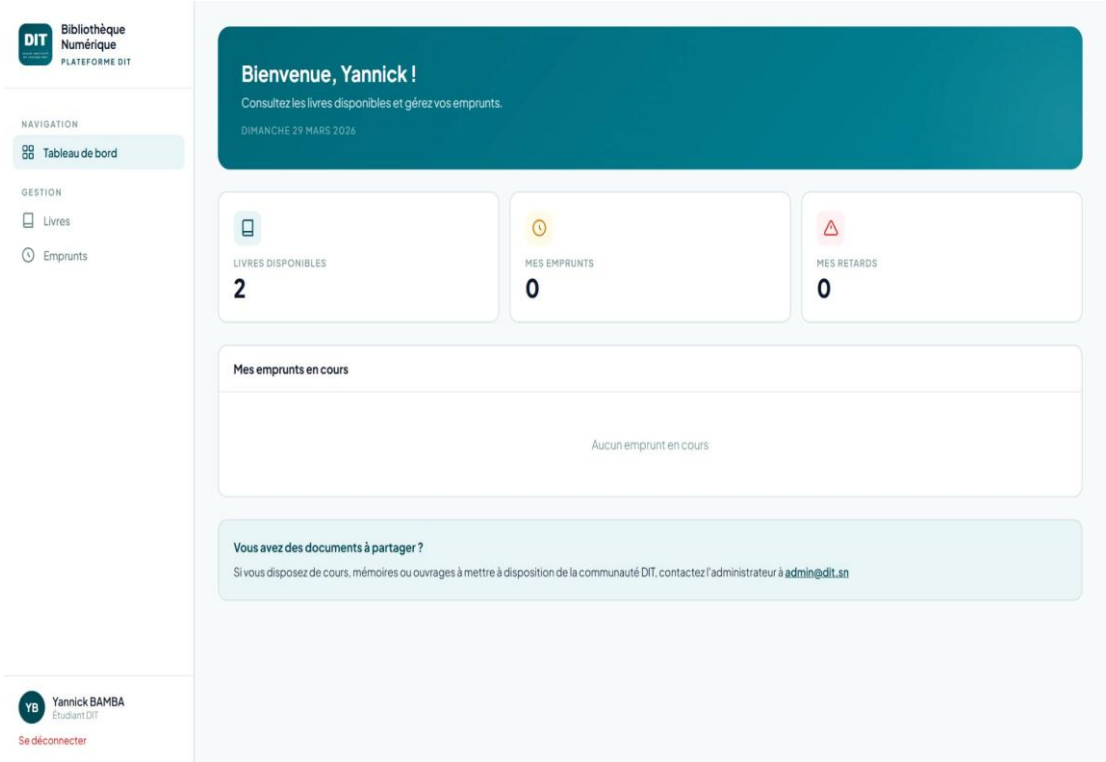


*Figure 7 – Dashboard membre avec emprunts personnels*

## 8.4 Gestion des livres

La page Livres affiche le catalogue complet avec une barre de recherche permettant de filtrer par titre, auteur ou ISBN. Chaque livre indique sa disponibilité en temps réel. L'administrateur dispose de fonctionnalités supplémentaires pour ajouter, modifier ou supprimer des livres. La disponibilité est mise à jour automatiquement après chaque emprunt ou retour validé.

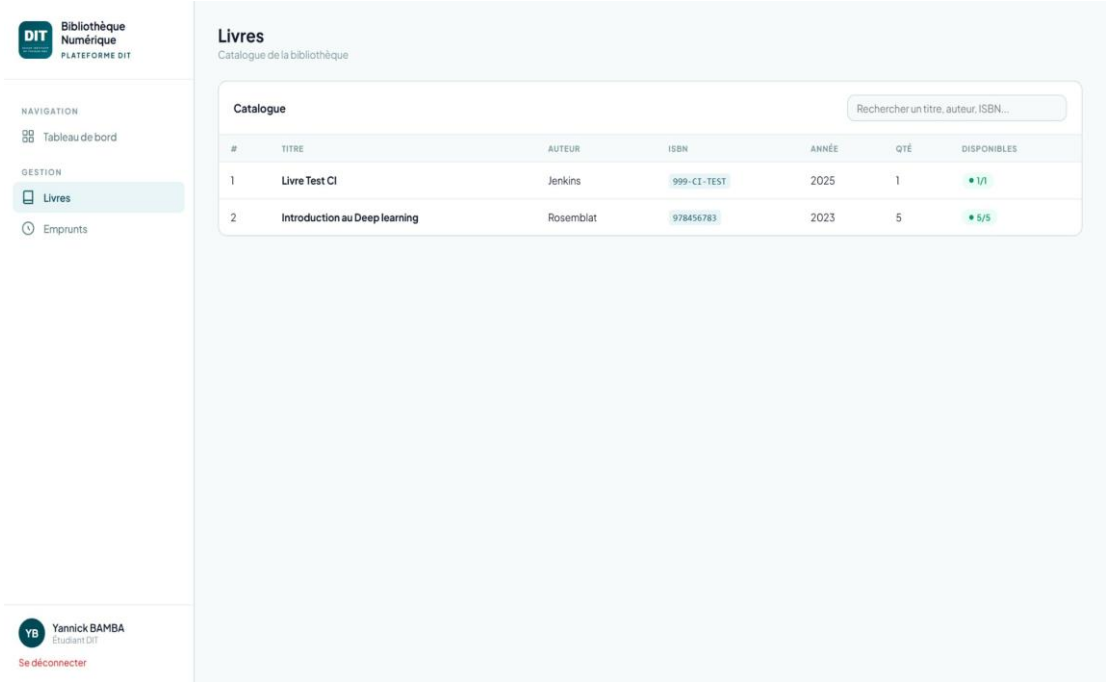


*Figure 8 – Catalogue des livres avec indicateurs de disponibilité*

## 8.5 Gestion des utilisateurs

La page Utilisateurs liste tous les membres inscrits. L'administrateur peut modifier les informations d'un membre ou désactiver son compte. La désactivation est bloquée si des emprunts sont en cours, avec un message explicite indiquant le nombre d'emprunts restants. Un compte désactivé conserve son historique complet d'emprunts.

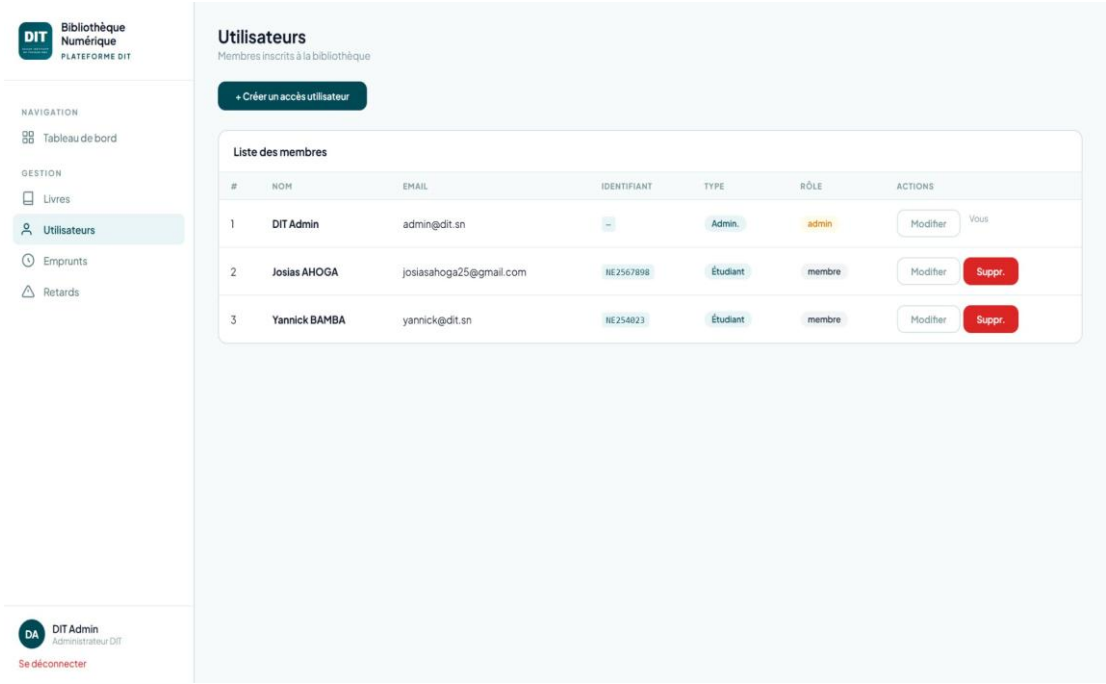


*Figure 9 – Liste des utilisateurs avec gestion des comptes*

## 8.6 Gestion des emprunts

La page Emprunts propose trois onglets avec compteurs en temps réel : En cours, En attente et Tous. L'onglet actif par défaut est « En cours ». L'onglet « En attente » affiche un badge d'alerte rouge lorsque des demandes attendent une validation de l'administrateur.

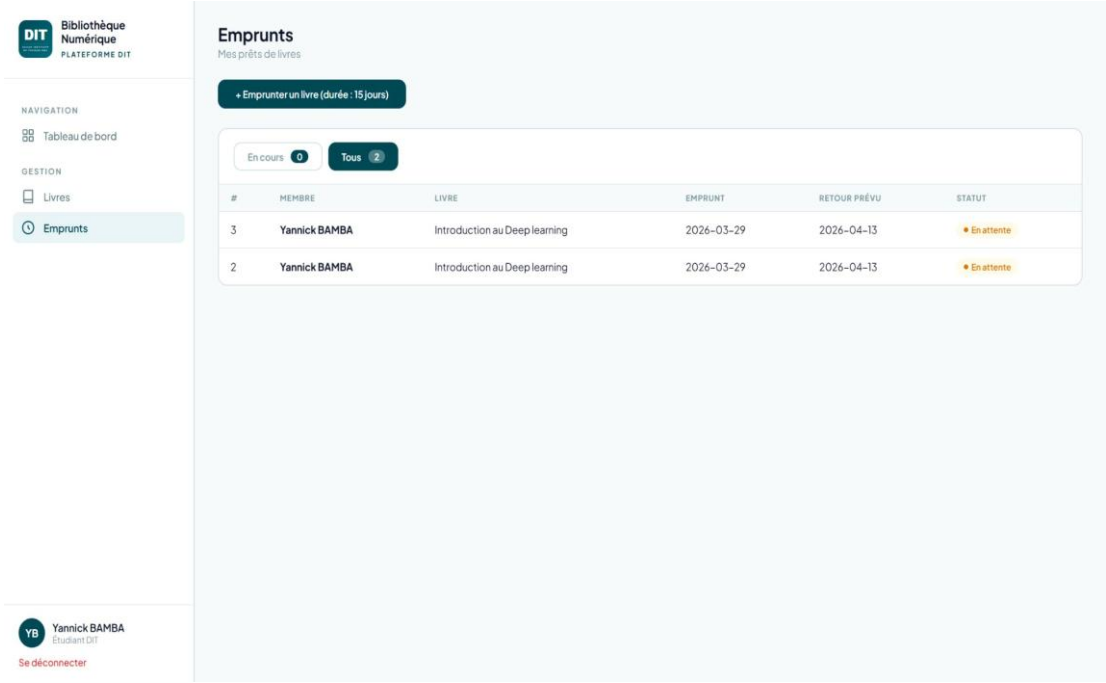


*Figure 10 – Page Emprunts avec les onglets et statuts*

## 8.7 Validation manuelle des emprunts

Lorsque la validation manuelle est activée (toggle désactivé sur le dashboard), les demandes d'emprunt sont créées avec le statut « En attente ». Le stock du livre n'est pas décrémenté tant que

l'administrateur n'a pas validé la demande. L'onglet « En attente » affiche un bouton « Valider » pour chaque demande en attente.

Sur le dashboard, la carte « En attente » affiche un badge rouge d'alerte et est cliquable pour accéder directement à l'onglet de validation.

## 8.8 Changement de mot de passe obligatoire

Lorsqu'un compte est créé par l'administrateur avec un mot de passe temporaire, l'utilisateur est accueilli par un modal bloquant à sa première connexion. Il est impossible de fermer ce modal ou d'accéder au tableau de bord sans avoir changé le mot de passe. Après validation, le flag mot_de_passe_temporaire passe à False en base de données et l'accès est accordé.

# 9. Docker et Docker Compose

Chaque composant possède son propre Dockerfile basé sur une image slim. Le fichier docker-compose.yml orchestre cinq conteneurs sur un réseau Docker dédié :

- **postgres :** image postgres:16-alpine avec healthcheck pg_isready.
- **service-livres :** FastAPI sur le port 8001, démarre après PostgreSQL.
- **service-utilisateurs :** FastAPI sur le port 8002, démarre après PostgreSQL.
- **service-emprunts :** FastAPI sur le port 8003, démarre après PostgreSQL et les deux autres services.
- **frontend :** Nginx Alpine sur le port 80, sert le fichier index.html unique.

Les commandes essentielles pour manipuler l'environnement sont les suivantes :

```
docker compose up --build          # Lancer l'application
docker compose down                # Arrêter l'application
docker compose logs --tail=50      # Consulter les logs
docker compose build service-emprunts  # Reconstruire un service
```

# 10. Pipeline CI/CD Jenkins

Le Jenkinsfile est conçu pour être compatible à la fois avec Windows et Linux grâce à la fonction isUnix(). En cas d'échec, le bloc post { failure } est protégé par un try/catch pour éviter les crashs liés à l'absence de shell sur Windows.

Le pipeline se décompose en sept étapes successives :

| Étape | Nom | Description |
|---|---|---|
| 1 | Récupération du code | Clone du dépôt GitHub. |
| 2 | Vérification de la structure | Contrôle de la présence des 12 fichiers essentiels. |
| 3 | Build des images Docker | Construction des 4 images Docker (avec diagnostic docker --version). |
| 4 | Arrêt de l'ancienne version | Suppression des anciens conteneurs (try/catch sur Windows). |
| 5 | Déploiement | Lancement des nouveaux conteneurs. |
| 6 | Vérification de santé | Test des endpoints /health des 3 services et du frontend. |
| 7 | Tests fonctionnels | Validation des APIs POST/GET et du flux de mot de passe temporaire. |

La capture ci-dessous montre l'exécution réussie du pipeline Jenkins avec toutes les étapes validées :

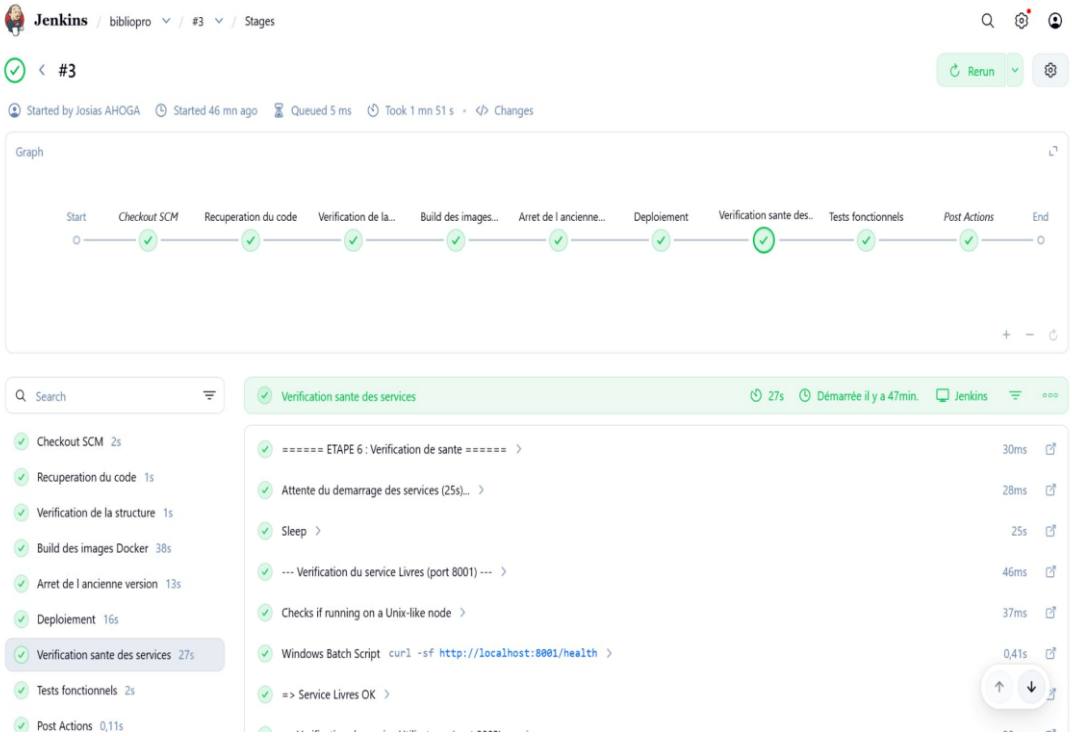


*Figure 11 – Pipeline Jenkins avec toutes les étapes validées*

# 11. Conclusion

Ce projet nous a permis de mettre en pratique les concepts DevOps fondamentaux : architecture microservices, conteneurisation Docker, orchestration Docker Compose, pipeline CI/CD Jenkins compatible multi-plateformes, et développement d'APIs REST avec FastAPI.

La plateforme couvre l'ensemble des besoins d'une bibliothèque académique avec des fonctionnalités avancées : gestion du catalogue avec suivi des exemplaires, authentification sécurisée par token, changement de mot de passe obligatoire à la première connexion, désactivation douce des comptes membres, système de validation des emprunts à double mode (automatique et manuel), et interface adaptée aux rôles avec notifications en temps réel.